



Comprehensive Learning Content: Inheritance

A deep dive into one of the most fundamental mechanisms in Object-Oriented Programming — from core concepts and memory layout to real-world use cases, design patterns, and best practices.

Topic Classification

Topic Name

Inheritance — Object-Oriented Programming / Software Design

Difficulty Level

Beginner to Advanced (progressive)

Topic Type

Core OOP Principle (Language Agnostic)

Prerequisites

- Classes & Objects
- Constructors
- Encapsulation
- Methods
- Basic Polymorphism

Industry Relevance

- Enterprise Software (Code Reuse)
- Framework/Library Design
- Game Development (Entity Hierarchies)
- GUI Frameworks (Widget hierarchies)
- ORM (Object-Relational Mapping)
- Design Patterns (Template Method, Strategy)

Recommended Learning Path

- 1

Classes & Objects Review
- 2

Inheritance Definition ("is-a")
- 3

Base & Derived Classes
- 4

Member Access (protected)
- 5

Constructor/Destructor Order
- 6

Method Overriding

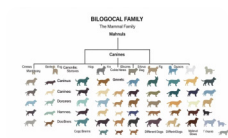
What Is Inheritance?

Inheritance is a fundamental mechanism in object-oriented programming where a new class (called **derived class**, **subclass**, or **child class**) is created from an existing class (called **base class**, **superclass**, or **parent class**). The derived class automatically **reuses (inherits)** all non-private members of the base class and can **extend** or **override** them. The relationship is described as "**is-a**" — e.g., a Dog **is an** Animal.



Family Genetics

Children inherit traits from parents. They can have unique traits (new methods) or modify inherited ones (override methods).



Biological Taxonomy

Mammal inherits from Animal.
Dog inherits from Mammal.
Poodle inherits from Dog.



Building Blueprints

A "house" blueprint inherits from "building" blueprint (walls, roof, foundation) and adds kitchen, bathroom, overrides roof style.



Job Roles

An Employee has name and ID. A Manager inherits and adds teamMembers. A Developer adds writeCode().

Core Purpose & Value



Code Reuse

Common code defined once in base class; reused across all derived classes



Extensibility

Add new features without modifying existing code (Open/Closed Principle)



Hierarchical Classification

Model real-world taxonomies naturally



Polymorphism

Treat derived class objects as base class objects

Why Does Inheritance Exist?

Problems It Solves

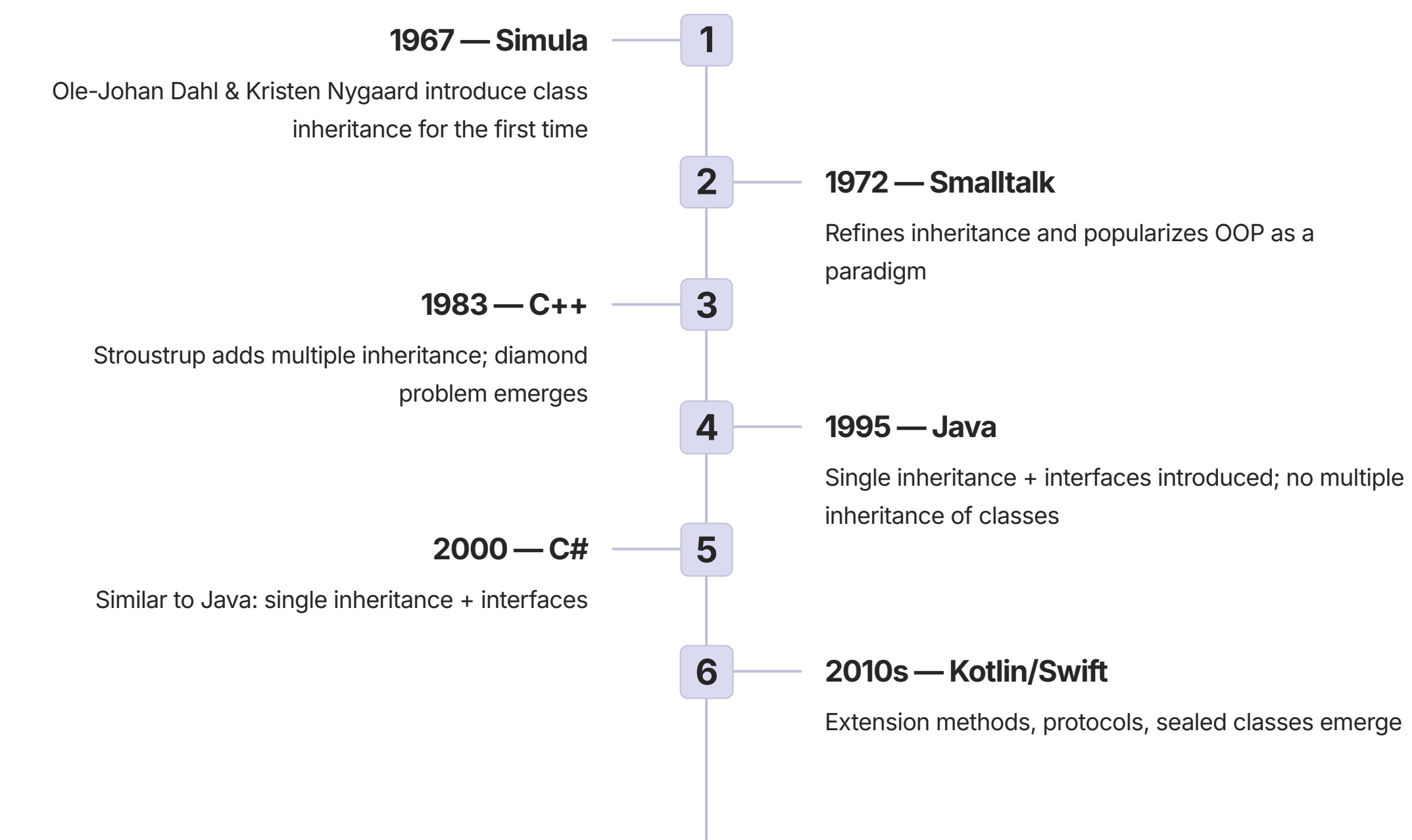
- **Code Duplication**
Common attributes/methods defined once in base class; shared by all derived classes
- **Maintenance Nightmare**
Change in one place (base class) propagates to all derived classes automatically
- **Inconsistent Behavior**
Base class ensures common behavior; derived classes only override what's different
- **Framework Extensibility**
Framework defines abstract base classes; users extend with concrete implementations

What Happens Without It

Scenario	Without Inheritance	Consequence
Animal classes	Dog, Cat, Bird each duplicate name, age, eat(), sleep()	4× code duplication; changing eat() requires editing 3 classes
GUI Framework	Each widget duplicates x, y, width, height, draw()	Massive duplication; no common event handling
Game Entities	Player, Enemy, NPC duplicate health, position, move()	Changes require editing all entity types

📌 Real-world motivation: GUI Frameworks (Java Swing, .NET WinForms), Game Engines (Unity MonoBehaviour), and Java Collections all rely heavily on inheritance for code reuse and polymorphism.

Evolution & History of Inheritance



Major Milestones

Milestone	Impact
Simula inheritance (1967)	First implementation of class inheritance
C++ multiple inheritance (1983)	Allowed inheriting from multiple base classes
Java interfaces (1995)	Solution to diamond problem; multiple inheritance of type
Liskov Substitution Principle (1994)	Formalized correctness rules for inheritance
Composition over inheritance (1994)	Gang of Four advocated preferring composition

Current Trends

Single Inheritance

Standard in Java, C#, Kotlin, Swift — remains dominant

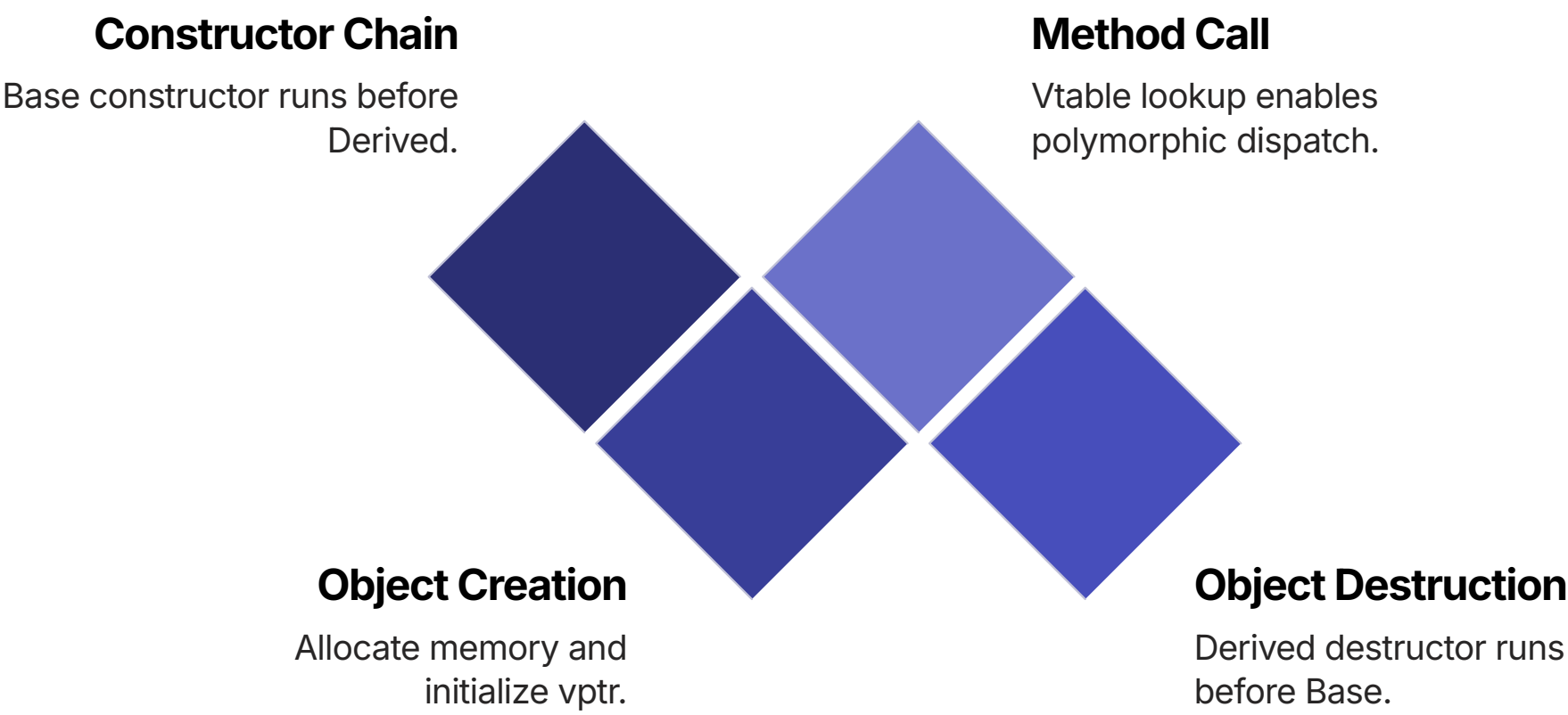
Composition Preference

Widely accepted best practice; inheritance used sparingly

Sealed Classes

Java 17 sealed classes; C# sealed; Kotlin sealed — more control over hierarchies

How Inheritance Works: Core Mechanisms



When a derived class object is created, memory is allocated for both the base sub-object and derived-specific members. The virtual pointer (vptr) is initialized to point to the correct vtable, enabling polymorphic dispatch at runtime. Constructors run from **Base → Derived**; destructors run in reverse: **Derived → Base**.

Access Modifiers in Inheritance

Modifier	Own Class	Derived Class	Outside
public	✓	✓	✓
protected	✓	✓	×
private	✓	×	×

⚠ Private members of the base class are **not accessible** in derived classes — not even with inheritance. Use `protected` for members intended for subclass access.

Virtual Dispatch (vtable)

- 01

Compiler sees call through base pointer

Detects method is virtual; generates vtable lookup code
- 02

Runtime fetches vptr from object

Hidden pointer in object points to class's vtable
- 03

Fetch function pointer from vtable slot

Dog's vtable contains `Dog::makeSound()`
- 04

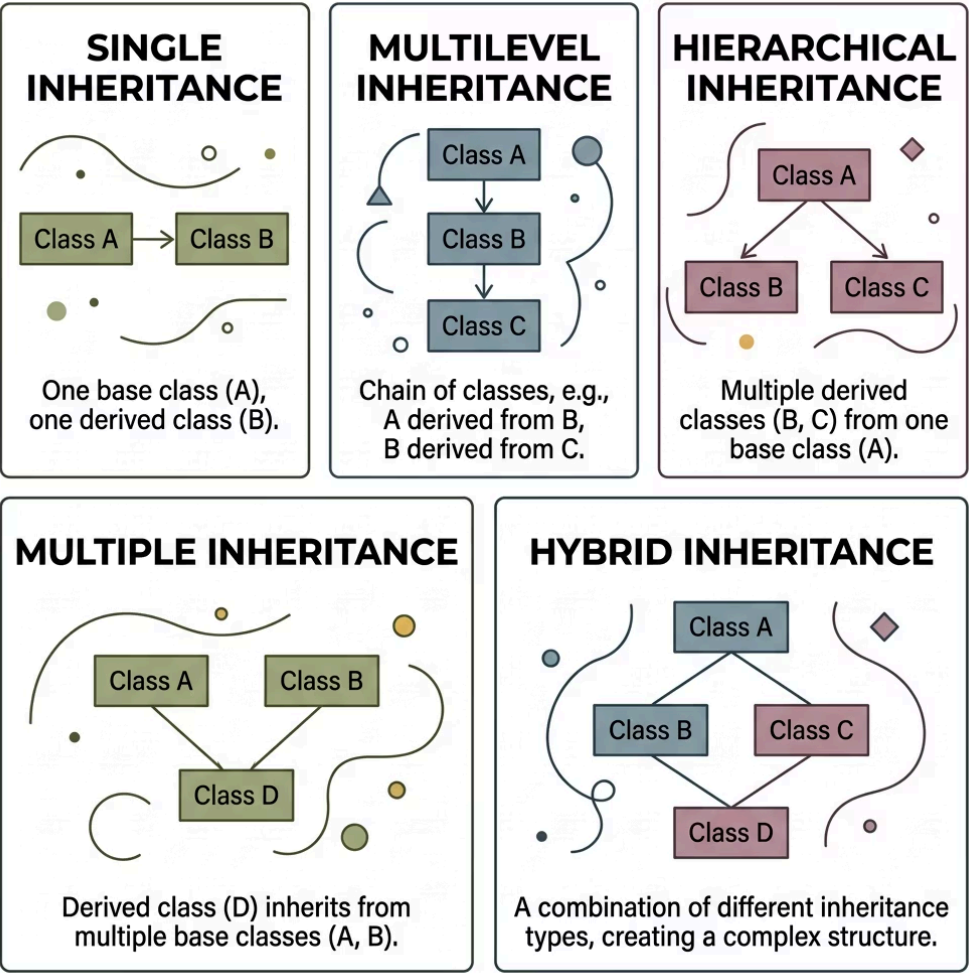
Correct overridden method executes

Dog's version runs even though variable type is `Animal*`

Types & Variants of Inheritance

Inheritance Types Compared

Type	Pattern	Language Support
Single	$A \rightarrow B$	All languages
Multilevel	$A \rightarrow B \rightarrow C$	All languages
Hierarchical	$A \rightarrow B, A \rightarrow C$	All languages
Multiple	$A \leftarrow D \rightarrow B$	C++, Python; Java/C# via interfaces
Hybrid	Mix of above	C++, Python



Method Override Categories

Concrete (non-virtual)

Cannot be overridden; method hiding only. Use for fixed utility methods.

Virtual

Can be overridden; base implementation exists. Use for customizable default behavior.

Abstract (Pure Virtual)

No implementation in base; **must** be overridden. Defines interface without implementation.

Sealed/Final

Cannot be overridden further. Prevents modification in subclasses.

Real-World Use Case: GUI Widget Hierarchy

GUI frameworks like **Java Swing** and **.NET WinForms** are a canonical example of inheritance in production. A **Component** abstract base class defines common properties (size, position, visibility) and methods shared by all widgets. Specialized widgets extend and implement their own **paint()** method.

Component (abstract)

x, y, width, height, visible, enabled
setLocation(), setSize(), setVisible(),
handleEvent()
paint() — *abstract*

Button

+ label, onClick
Implements paint(), click()

TextField

+ text, editable
Implements paint(), setText()

CheckBox

+ checked, onCheck
Implements paint(), toggle()

Window

+ title, menuBar
Implements paint(), add()

✅ **Outcome:** Millions of applications built on Swing/WinForms; common event system; uniform layout management. Every widget inherits position, size, and visibility — each only implements its own **paint()**.

📌 **Lesson Learned:** Deep inheritance (Component → Container → JComponent → JPanel) can become complex. Composition is often preferred for complex widgets.

Real-World Use Case: Game Entity Hierarchy

The Problem

All game entities — Player, Enemy, NPC, Item — need common behavior: position, health, update, render. The game loop must call `update()` and `render()` on every entity polymorphically.

The Solution

An abstract `Entity` base class defines common fields and virtual methods. Derived classes override `update()` and `render()` with their specific logic.

```
// Game loop (polymorphic)
List<Entity> entities = getAllEntities();
foreach (Entity e in entities) {
    e.update(deltaTime);
    e.render();
}
```

✅ **Benefit:** Add new enemy types without modifying the game loop. Polymorphism handles dispatch automatically.

⚠️ **Lesson:** Prefer composition (component-based architecture) over deep inheritance for game entities. Inheritance depth > 3 can become rigid.

Entity (abstract)

position: Vector2, health: float, isActive: bool

update(deltaTime) — virtual/abstract

render() — virtual/abstract

takeDamage(), move()

Player

+ input, score

update(), handleInput()

Enemy

+ aiState, target

update(), chase()

NPC

+ dialogue, questId

update(), interact()

Item

+ pickup, effect

update(), onPickup()

Example 1: Basic Single Inheritance

The classic Animal hierarchy demonstrates inheritance fundamentals: calling `SUPER()` in constructors, adding new methods, overriding virtual methods, and polymorphic dispatch through a base class reference.

```
CLASS Animal:
  PROTECTED:
    name: STRING
    age: INTEGER

  CONSTRUCTOR(name, age):
    this.name = name
    this.age = age

  FUNCTION eat():
    PRINT name + " is eating"

  FUNCTION makeSound(): // virtual
    PRINT name + " makes a generic sound"

CLASS Dog EXTENDS Animal:
  PRIVATE:
    breed: STRING

  CONSTRUCTOR(name, age, breed):
    SUPER(name, age) // call parent FIRST
    this.breed = breed

  FUNCTION bark():
    PRINT name + " barks: Woof!"

  OVERRIDE FUNCTION makeSound():
    PRINT name + " barks: Woof woof!"

CLASS Cat EXTENDS Animal:
  CONSTRUCTOR(name, age):
    SUPER(name, age)

  OVERRIDE FUNCTION makeSound():
    PRINT name + " meows: Meow!"
```

```
// Usage:
dog = Dog("Rex", 3, "Labrador")
cat = Cat("Whiskers", 2)
bird = Bird("Tweety", 1, 0.25)

// Inherited methods
dog.eat()    // "Rex is eating"
dog.sleep()  // "Rex is sleeping"

// New methods
dog.bark()   // "Rex barks: Woof!"

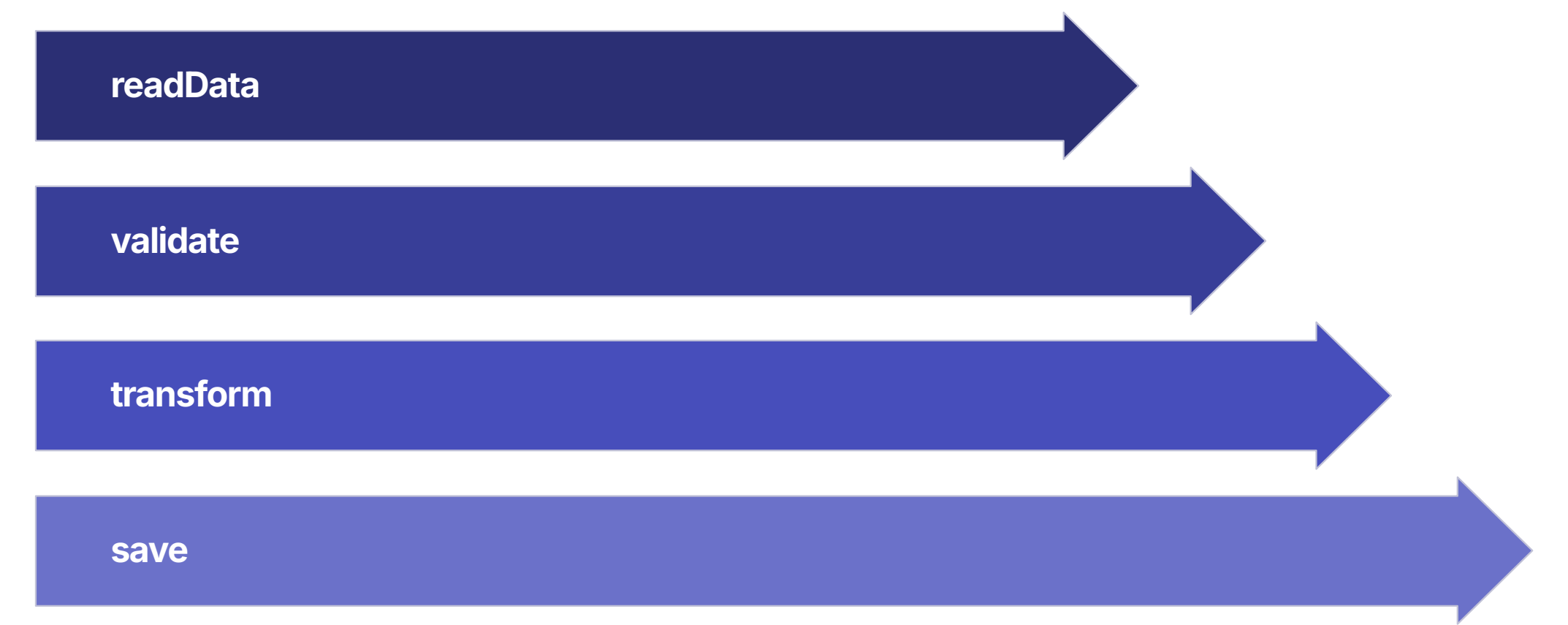
// Overridden methods
dog.makeSound() // "Rex barks: Woof woof!"
cat.makeSound() // "Whiskers meows: Meow!"
bird.makeSound()// "Tweety makes a generic sound"

// Polymorphism (base class reference)
animals = [dog, cat, bird]
FOR each a IN animals:
  a.makeSound()
  // Each calls its OWN version!
```

👍 **Key Insight:** The polymorphic loop calls each animal's own `makeSound()` — this is dynamic dispatch in action. The variable type is `Animal`, but the runtime type determines which method runs.

Example 2: Abstract Class — Template Method Pattern

The **Template Method Pattern** is a classic inheritance-based design pattern. The abstract base class defines the algorithm's skeleton; derived classes implement specific steps without changing the overall structure.



```
ABSTRACT CLASS DataProcessor:
    // Template method (algorithm skeleton)
    FUNCTION process():
        data = this.readData()    // abstract
        IF NOT this.validate(data):
            RAISE ERROR
        processed = this.transform(data) // abstract
        this.save(processed)        // abstract
        this.logCompletion()        // concrete/final
        RETURN processed

    ABSTRACT FUNCTION readData()
    ABSTRACT FUNCTION transform(data)
    ABSTRACT FUNCTION save(data)

    FUNCTION validate(data):
        RETURN data != NULL // default

    FUNCTION logCompletion():
        PRINT "Completed at " + CURRENT_TIME()
```

```
CLASS CSVProcessor EXTENDS DataProcessor:
    CONSTRUCTOR(filename):
        this.filename = filename

    OVERRIDE FUNCTION readData():
        PRINT "Reading CSV from " + filename
        RETURN ["row1", "row2", "row3"]

    OVERRIDE FUNCTION transform(data):
        RETURN data + ["transformed"]

    OVERRIDE FUNCTION save(data):
        PRINT "Saving to CSV file"

CLASS JSONProcessor EXTENDS DataProcessor:
    OVERRIDE FUNCTION readData():
        PRINT "Fetching JSON from " + url
        RETURN {"key": "value"}

    OVERRIDE FUNCTION validate(data):
        RETURN data != NULL AND data HAS KEY "key"

    OVERRIDE FUNCTION save(data):
        PRINT "Sending JSON to API"

// Both use same algorithm, different implementations
csvProc.process()
jsonProc.process()
```

Example 3: Multiple Inheritance & the Diamond Problem

Multiple Inheritance (Conceptual)

```
CLASS Printable:
  FUNCTION print():
    PRINT "Printable: " + this.toString()
  ABSTRACT FUNCTION toString() -> STRING

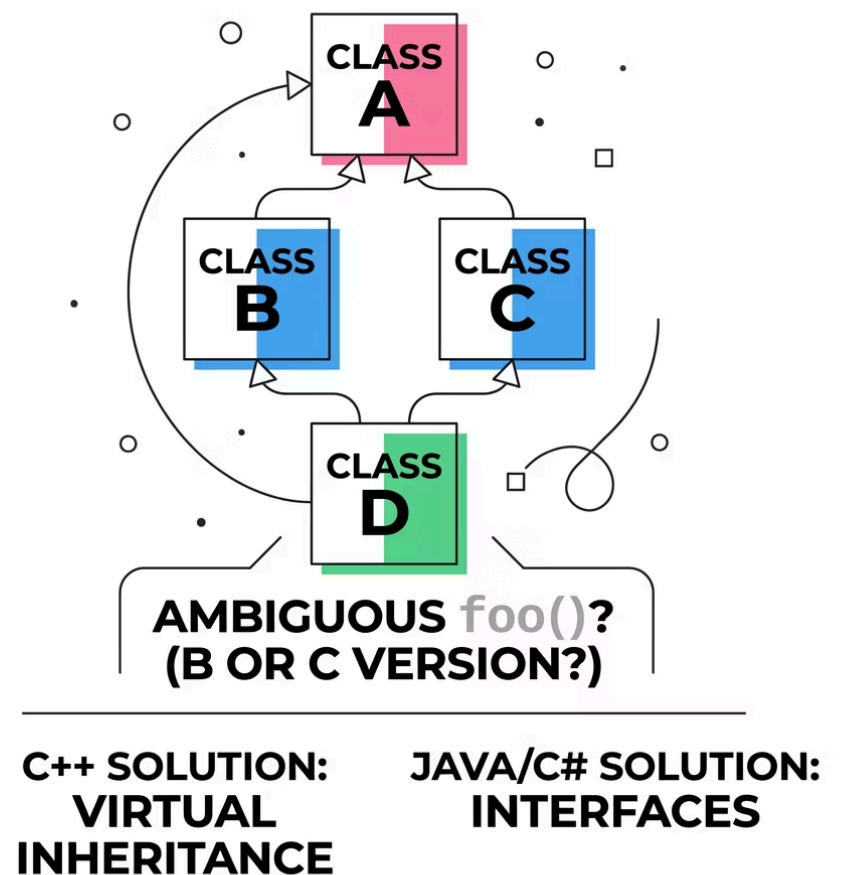
CLASS Serializable:
  FUNCTION serialize():
    RETURN "Serialized: " + this.toData()
  ABSTRACT FUNCTION toData() -> STRING

// Inheriting from TWO base classes
CLASS Document EXTENDS Printable, Serializable:
  PRIVATE:
    title: STRING
    content: STRING

  OVERRIDE FUNCTION toString():
    RETURN "Document: " + this.title

  OVERRIDE FUNCTION toData():
    RETURN this.title + "|" + this.content
```

The Diamond Problem



⚠ Diamond Problem: Without virtual inheritance (C++), class D would have two copies of A's members and an ambiguous `foo()` call. C++ solves with `virtual` inheritance. Java/C# avoid this entirely by using interfaces instead of multiple class inheritance.

Hands-On Lab 1: Shape Hierarchy

BEGINNER LAB

Objective: Create an abstract Shape class and implement Rectangle, Circle, and Triangle subclasses with polymorphic area and perimeter calculations.

Step 1 — Abstract Shape Class

```
ABSTRACT CLASS Shape:
  PROTECTED:
    color: STRING

  CONSTRUCTOR(color: STRING):
    this.color = color

  ABSTRACT FUNCTION area() -> DOUBLE
  ABSTRACT FUNCTION perimeter() -> DOUBLE

  FUNCTION getColor() -> STRING:
    RETURN this.color
```

Step 2 — Rectangle

```
CLASS Rectangle EXTENDS Shape:
  PRIVATE:
    width: DOUBLE
    height: DOUBLE

  CONSTRUCTOR(color, width, height):
    SUPER(color)
    this.width = width
    this.height = height

  OVERRIDE FUNCTION area():
    RETURN this.width * this.height

  OVERRIDE FUNCTION perimeter():
    RETURN 2 * (this.width + this.height)
```

Step 3 — Circle & Triangle

```
CLASS Circle EXTENDS Shape:
  PRIVATE:
    radius: DOUBLE

  CONSTRUCTOR(color, radius):
    SUPER(color)
    this.radius = radius

  OVERRIDE FUNCTION area():
    RETURN PI * radius * radius

  OVERRIDE FUNCTION perimeter():
    RETURN 2 * PI * radius

CLASS Triangle EXTENDS Shape:
  PRIVATE:
    sideA, sideB, sideC: DOUBLE

  OVERRIDE FUNCTION area():
    // Heron's formula
    s = (sideA + sideB + sideC) / 2
    RETURN SQRT(s*(s-sideA)*(s-sideB)*(s-sideC))

  OVERRIDE FUNCTION perimeter():
    RETURN sideA + sideB + sideC
```

Step 4 — Test Polymorphism

```
shapes = [
  Rectangle("Red", 5.0, 3.0),
  Circle("Blue", 4.0),
  Triangle("Green", 3.0, 4.0, 5.0)
]

FOR each shape IN shapes:
  PRINT shape.getColor()
  PRINT "Area:", shape.area()
  PRINT "Perimeter:", shape.perimeter()
```

✔ **Learning Outcome:** Understand abstract classes, method overriding, and polymorphism through base class references. Total area of all shapes: **71.265**

Hands-On Lab 2: Employee Payroll System

INTERMEDIATE LAB

Objective: Implement different employee types (Salaried, Hourly, Commission) with polymorphic pay calculation. The `PayrollSystem` works with any `Employee` subclass.

Employee (abstract)
name: STRING, employeeId: STRING
calculatePay() → DOUBLE (abstract)
getName(), getEmployeeId(), to_string()

SalariedEmployee
annualSalary: DOUBLE
calculatePay() = annualSalary / 26.0
Bi-weekly pay periods

HourlyEmployee
hourlyRate, hoursWorked: DOUBLE
calculatePay() with 1.5× overtime over 80 hrs
addHours(), resetHours()

CommissionEmployee
baseSalary, commissionRate, salesAmount
calculatePay() = baseSalary + (sales × rate / 100)
addSales(), resetSales()

CLASS PayrollSystem:
 PRIVATE:
 employees: LIST OF Employee

 FUNCTION addEmployee(emp: Employee):
 this.employees.append(emp)

 FUNCTION processPayroll():
 totalPay = 0.0
 FOR each emp IN this.employees:
 pay = emp.calculatePay()
 totalPay += pay
 PRINT emp.to_string() + " - Pay: \$" + pay
 PRINT "Total payroll: \$" + totalPay

// Expected Output:
// === PAYROLL REPORT ===
// E001: Alice Johnson - Pay: \$3000.00
// E002: Bob Smith - Pay: \$2187.50
// E003: Carol Davis - Pay: \$4500.00
// Total payroll: \$9687.50

 **Learning Outcome:** Each employee type implements `calculatePay()` differently. The payroll system calls it polymorphically — it doesn't need to know which type it's dealing with.

Advantages & Disadvantages

✓ Advantages

- **Code Reuse:** Common code in base class; no duplication across derived classes
- **Polymorphism:** Write code that works with base class; works with any derived class
- **Extensibility:** Add new derived classes without modifying existing code (Open/Closed)
- **Maintainability:** Bug fix in base class fixes all derived classes simultaneously
- **Logical Modeling:** Models "is-a" relationships naturally and intuitively
- **Framework Design:** Abstract base classes define contracts for extensibility

✗ Disadvantages

- **Brittle Base Class:** Changes in base class affect all derived classes unexpectedly
- **Virtual Dispatch Overhead:** Small but non-zero cost for polymorphic calls
- **Rigid Hierarchies:** Deep hierarchies become hard to change or refactor
- **Tight Coupling:** Inheritance couples derived classes tightly to base class
- **Forced Hierarchies:** Can force unnatural "is-a" relationships
- **Fragile Design:** Overuse leads to fragile, unmaintainable hierarchies

Performance Considerations

Complexity Analysis

Operation	Time Complexity	Notes
Non-virtual method call	$O(1)$ — direct call	Same as regular function
Virtual method call	$O(1)$ — vtable indirection	One extra dereference (negligible)
Constructor chain	$O(\text{depth of hierarchy})$	Each constructor in chain runs
Dynamic cast	$O(1)$ to $O(\text{depth})$	C++ <code>dynamic_cast</code> ; Java <code>instanceof</code>
Memory overhead per object	1 pointer (vptr)	8 bytes on 64-bit systems

Common Bottlenecks

Bottleneck	Cause	Mitigation
Deep virtual call chains	Many levels of overridden methods	Keep hierarchy shallow (< 4 levels)
Constructor overhead	Each level initializes fields	Avoid complex logic in constructors
Dynamic casting	Runtime type checks	Use polymorphism instead of type switches

Optimization Techniques

Final/Sealed Classes

Prevent overriding → enables compiler devirtualization (direct call instead of vtable lookup)

Compiler Devirtualization

Detect concrete type at compile time → eliminate vtable lookup entirely

Inlining Virtual Calls

Small, frequently called methods → reduces call overhead significantly

Shallow Hierarchies

Limit inheritance depth → faster construction and less complexity

 Virtual dispatch overhead is typically **negligible** in modern CPUs due to branch prediction and CPU caches. Optimize only when profiling confirms it's a bottleneck.

Scalability & Reliability

Scaling Strategies

Scale	Consideration	Strategy
Hierarchy depth	2–4 levels optimal	Flatten hierarchy; prefer composition
Number of derived classes	Hundreds possible	Base class must be stable; document properly
Framework size	1000+ classes	Sealed classes where appropriate; interfaces for contracts

Liskov Substitution Principle — Critical Example

```
// LSP VIOLATION: Square extends Rectangle
class Square extends Rectangle {
  void setWidth(w) {
    width = w;
    height = w; // Violates LSP!
  }
}

// Code that works with Rectangle BREAKS with Square:
void resize(Rectangle r) {
  r.setWidth(5);
  r.setHeight(10);
  // Expect area = 50
  // But Square gives area = 100 ← WRONG!
}
```

Reliability Patterns

Liskov Substitution Principle (LSP)

Derived class must be substitutable for base. Preconditions cannot be strengthened; postconditions cannot be weakened; invariants must be preserved.

Template Method Pattern

Base class defines algorithm structure. Derived classes implement specific steps. Prevents code duplication.

Fragile Base Class Problem

Changes to base class can break derived classes. Mitigation: document "override" contracts; use composition over inheritance.

Sealed/Final Classes

Mark classes as sealed when inheritance is not intended. Prevents misuse and unintended subclassing.

Design & Architectural Considerations

Key Architecture Decisions

Decision Point	Recommendation
Inheritance vs Composition	Prefer composition; use inheritance only for true "is-a"
Depth of hierarchy	Keep shallow (≤ 4 levels)
Abstract vs Concrete base	Use abstract base classes for frameworks
Multiple inheritance	Use interfaces; avoid multiple inheritance of state
Virtual vs Non-virtual	Mark only methods meant for override as virtual
Protected members	Minimal; prefer private + getters/setters

Trade-offs: Inheritance vs Composition

Criteria	Inheritance	Composition
Relationship	"is-a"	"has-a"
Binding	Compile-time (static)	Runtime (dynamic)
Flexibility	Low (rigid)	High (change at runtime)
Coupling	Tight	Loose
Change impact	Base changes affect all derived	Only if interface changes
Testing	Harder (can't easily mock base)	Easier (inject mocks)
GoF advice	Use sparingly	"Favor composition"

Inheritance-Based Design Patterns

Template Method

Base defines algorithm skeleton; derived implement specific steps

Factory Method

Base defines factory method; derived create specific objects

Strategy

Abstract strategy base; concrete strategies as derived classes

Command

Abstract command base; concrete commands as derived classes

Security Considerations

Threat Model

Threat	Description	Risk
Base class poisoning	Malicious derived class breaks base invariants	MEDIUM
Finalize attacks	Overriding finalize to resurrect objects	LOW (Java)
Type confusion	Casting vulnerabilities	LOW (type-safe languages)
Privilege escalation	Inheriting from privileged base class	MEDIUM
Reflection access	Accessing protected members via reflection	MEDIUM

Common Vulnerabilities

Vulnerability	Mitigation
Calling overridable methods in constructor	Never call overridable methods in constructor
Breaking invariants in subclass	Document invariants; seal class if needed
Exposing protected fields	Keep fields private; provide protected methods
Serialization of inherited classes	Validate in readObject()

Hardening Guidelines

Declare classes final/sealed

Unless designed for inheritance — prevents unintended subclassing

Document override contracts

Clearly state which methods are meant to be overridden and what invariants must be preserved

Make fields private, not protected

Prevents direct access from subclasses; use protected methods instead

Never call overridable methods in constructor

Avoids partial initialization — derived class state not yet set up

Use interfaces over abstract classes

Reduces coupling; prevents unintended state sharing



Calling an overridable method in a constructor is one of the most dangerous inheritance mistakes. The derived class constructor hasn't run yet, so the overridden method may access uninitialized state.

Best Practices

✓ DO's

Use inheritance only for "is-a" relationships

The derived class must genuinely be a specialization of the base class

Ensure Liskov Substitution

Derived class must be fully substitutable for base class without breaking behavior

Keep hierarchies shallow (≤ 4 levels)

Deep hierarchies become rigid and hard to maintain

Use abstract base classes to define contracts

Especially in framework design; forces derived classes to implement required behavior

Mark classes sealed/final unless designed for inheritance

Prevents misuse and unintended subclassing

✗ DON'Ts

Don't create hierarchies deeper than 4–5 levels

Leads to rigid, unmaintainable code that's hard to refactor

Never call overridable methods in constructor

Polymorphism hazard — derived class not yet initialized

Avoid protected fields

Use private + protected methods instead to maintain encapsulation

Don't use inheritance purely for code reuse

Prefer composition when "is-a" is false — use "has-a" instead

Avoid downcasting

Use polymorphism instead of casting to derived types

Common Mistakes & Pitfalls

1

Beginner Mistakes

Forgetting `super()` call; confusing overriding with overloading; making fields `protected` by default; using inheritance for code reuse only

2

Intermediate Mistakes

LSP violation (Square extends Rectangle); deep inheritance trees; calling overridable method in constructor; not marking methods as `final` when appropriate

3

Advanced / Production Mistakes

Fragile base class problem; inheritance for DTOs; multiple inheritance misuse (diamond problem); breaking LSP with strengthened preconditions

Mistake	Why It Happens	Impact	Correct Approach
Forgetting <code>super()</code>	Doesn't know parent constructor required	Compile error	Call <code>super(params)</code> first in constructor
Confusing override with overload	Different parameter signature	Base version called instead	Use <code>@Override</code> (Java) or <code>override</code> keyword
LSP violation (Square/Rectangle)	Seems logical but breaks	Unexpected behavior	Favor composition or separate hierarchy
Calling overridable method in constructor	Unaware of initialization order	Subclass state not initialized	Never call; use factory method pattern
Fragile base class	Changing base breaks derived	Widespread failures	Document contracts; prefer composition

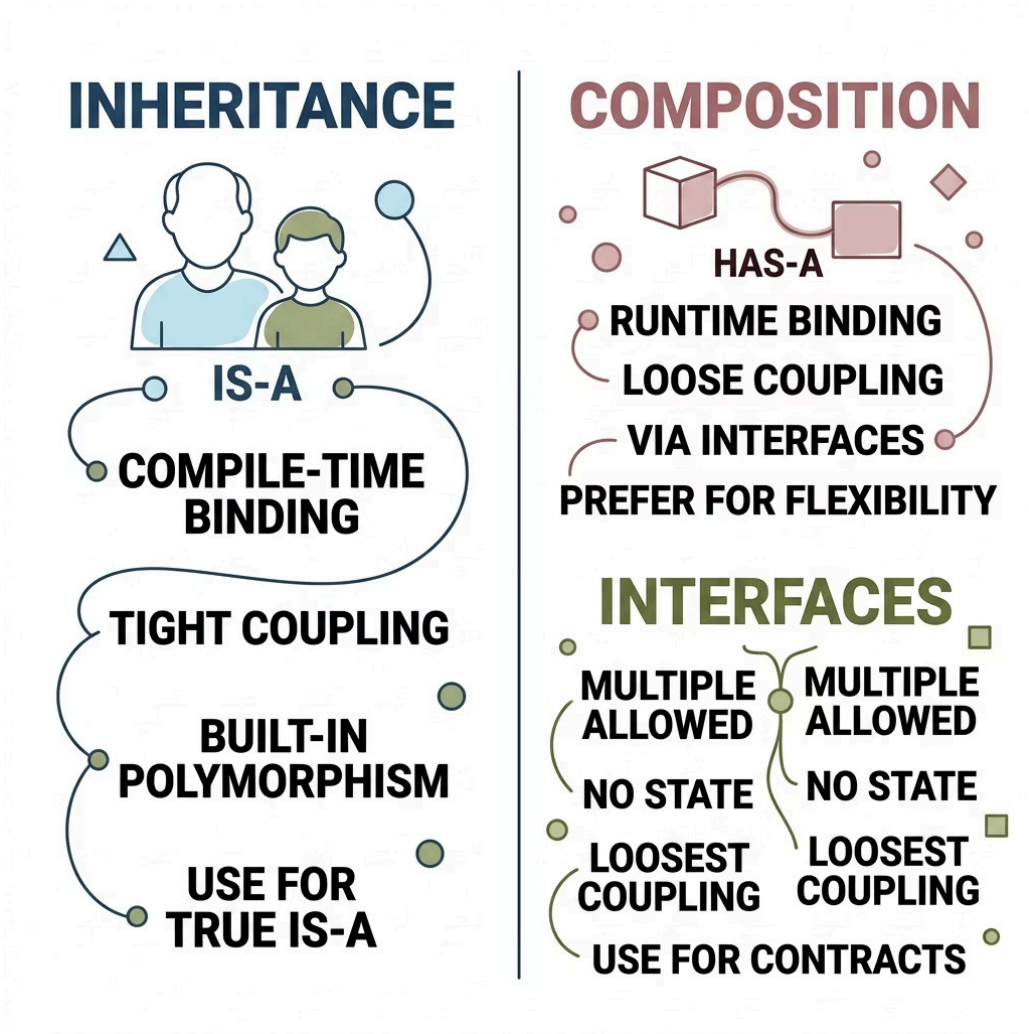
Comparison with Alternatives

Inheritance vs Interfaces

Criteria	Inheritance (Class)	Interface
Multiple	Single in Java/C#, multiple in C++	Yes (multiple implementation)
Code reuse	Yes (inherits implementation)	No (only contract)
State	Can have fields	No fields (constants only)
Constructors	Yes	No
Coupling	Tighter	Looser
When to use	"is-a" + shared implementation	"can-do" capability

Language Feature Comparison

Feature	Java	C++	C#	Python	Kotlin
Multiple inheritance	No	Yes	No	Yes	No
Abstract classes	abstract	Pure virtual	abstract	ABC module	abstract
Sealed/Final	sealed/final	final	sealed	convention	sealed
Virtual by default?	Yes	No	Yes	Yes	Yes



Learning Summary: Key Takeaways



Inheritance

Derived class acquires members of base class. The "is-a" relationship is essential.



Liskov Substitution

Derived must be substitutable for base without breaking code. Preconditions cannot be strengthened.



Method Overriding

Derived class provides new implementation of base method. Runtime decides which version runs.



Constructor Order

Base constructor runs FIRST, then derived. Destructors run in reverse order.



Abstract Class

Cannot be instantiated; exists only to be inherited. Defines contracts for derived classes.



Composition Preference

Often preferred over inheritance. More flexible; loosely coupled; easier to test.

Memory Aids

“

"Base first, then child — base builds the foundation."

Constructor order

“

"If it looks like a duck and quacks like a duck, it should behave like a duck."

Liskov Substitution Principle

“

"Is-a for hierarchy; Has-a for assembly."

Inheritance vs Composition

“

"Runtime decides, not compile time."

Virtual dispatch

”

”

Quick Reference Cheat Sheet

Inheritance Syntax (Pseudo)

```
class Derived extends Base {  
    constructor(params) {  
        super(baseParams); // call base FIRST  
        // derived fields  
    }  
  
    @Override methodName() {  
        // new behavior  
    }  
}
```

Constructor / Destructor Order

Construction:

Base → Derived(fields) → Derived(body)

Destruction:

Derived(body) → Derived(fields) → Base

Access Modifiers

public → accessible everywhere
protected → accessible in class + subclasses
private → accessible only in own class
package → accessible in same package (Java)

Liskov Substitution Principle (LSP)

- **Preconditions:** cannot be strengthened
- **Postconditions:** cannot be weakened
- **Invariants:** must be preserved
- **History:** derived can't change state allowed by base

Inheritance vs Composition

Inheritance: "is-a" (Dog **is** an Animal)

Composition: "has-a" (Car has an Engine)

Prefer composition unless **true** "is-a"

Virtual Dispatch (vtable)

```
Animal* a = new Dog();  
a->makeSound();  
// → fetches Dog's vtable  
// → calls Dog::makeSound()  
// NOT Animal::makeSound()
```

 Use `@Override` (Java) or `override` keyword (C++/C#/Kotlin) to catch accidental method hiding at compile time.

Common Interview & Certification Questions

BEGINNER

- 01
- What is inheritance? What is the "is-a" relationship?
- 02
- Difference between method overriding and method overloading?
- 03
- What is the purpose of super() in a constructor?
- 04
- Can a derived class access private members of base class? Why?
- 05
- What is the order of constructor execution in inheritance?

INTERMEDIATE

- 01
- Explain Liskov Substitution Principle with an example.
- 02
- What is the diamond problem? How do different languages solve it?
- 03
- Difference between abstract class and interface. When to use each?
- 04
- Why is calling overridable methods in constructor a problem?
- 05
- What are the trade-offs between inheritance and composition?

ADVANCED / EXPERT

- 01
- How does virtual function dispatch work? Explain vtable concept.
- 02
- What is the fragile base class problem? How to mitigate?
- 03
- Explain the template method pattern and its relationship to inheritance.
- 04
- How does multiple inheritance work in C++? What is virtual inheritance?
- 05
- Why does Java not support multiple inheritance of classes? What alternative does it provide?

Pro tip: For LSP questions, always use the Square/Rectangle example. For diamond problem, explain how Java interfaces solve it. For vtable, walk through the 4-step dispatch process.

Recommended Next Topics

Having mastered Inheritance, these topics form the natural progression of your OOP and software design knowledge:



Polymorphism

Natural extension of inheritance. Dynamic dispatch, method overriding, and runtime type resolution in depth.



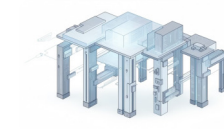
Abstract Classes & Interfaces

Advanced abstraction mechanisms. When to use abstract classes vs interfaces; default methods; multiple interface implementation.



Composition over Inheritance

Design principle for when to prefer composition. Delegation patterns; dependency injection; flexible runtime behavior.



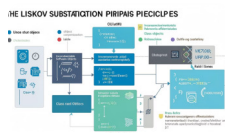
SOLID Principles

Broader design principles including Open/Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion.



Design Patterns (GoF)

Template Method, Factory Method, Strategy, Command, State — all leverage inheritance as a core mechanism.



Liskov Substitution Principle

Formal correctness rules for inheritance. Preconditions, postconditions, invariants, and behavioral subtyping in depth.

End of Document

Inheritance Mastered

You have completed the comprehensive learning content on **Inheritance** — from formal definitions and memory layout internals, through real-world use cases in GUI frameworks and game engines, to hands-on labs, security considerations, and design best practices.

10

Core Sections

Topic classification through learning summary

4

Code Examples

Basic inheritance, template method, multiple inheritance, sealed classes

2

Hands-On Labs

Shape hierarchy (beginner) and employee payroll system (intermediate)

15

Interview Questions

Beginner, intermediate, and advanced/expert levels covered

- ✓ **Key Principle to Remember:** Use inheritance for true "is-a" relationships. Ensure Liskov Substitution. Keep hierarchies shallow. When in doubt, prefer composition over inheritance — it's more flexible, loosely coupled, and easier to test.

